LLM-Powered Multi-Agent Systems: A Technical Framework for Collaborative Intelligence through Optimized Knowledge Retrieval and Communication

Vineeth Gogineni
Columbia Business School
Columbia University
NewYork, United States
goginenivineeth@gmail.com

2025 6th International Conference on Artificial Intelligence, Robotics and Control (AIRC) | 979-8-3315-4348-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/AIRC64931.2025.11077480

***Abstract*—This paper presents a comprehensive technical framework for constructing effective multi-agent systems powered by large language models (LLMs). We examine how multiple LLM-based agents can collaborate to solve complex problems beyond the capabilities of single agents through specialized knowledge integration and optimized communication protocols. Our novel architecture enables efficient collaboration between heterogeneous LLM agents, each with domain-specific capabilities and knowledge bases. Experimental results demonstrate that our multi-agent LLM system achieves 42% higher accuracy on complex knowledge tasks, 37% reduction in hallucinations, and 29% faster convergence on collaborative problem-solving compared to baseline approaches. By implementing optimized knowledge retrieval mechanisms and structured communication patterns, our system reduces token usage by 45% while maintaining semantic fidelity across agent interactions. These findings establish key design principles for building more effective and reliable collaborative LLM-based multi-agent systems for enterprise applications**

***Index Terms*—Large language models, multi-agent systems, retrieval-augmented generation, vector databases, prompt engineering, knowledge retrieval**

I. Introduction

Recent advances in large language models (LLMs) have created unprecedented opportunities for building sophisticated multi-agent systems capable of complex problem solving through collaboration. Unlike traditional multi-agent architectures that rely on predefined rules, modern LLM-based agents can leverage their emergent capabilities for flexible communication, task decomposition, and knowledge integration [1]. However, significant challenges remain in optimizing knowledge retrieval across agents, ensuring coherent reasoning, and maintaining factual accuracy in collaborative settings.

This research addresses three critical technical challenges in LLM-based multi-agent systems:

1) How can Retrieval-Augmented Generation (RAG) be effectively integrated in a multi-agent context to ensure knowledge consistency and accuracy?
2) What prompt engineering techniques optimize inter-agent communication while minimizing token usage?
3) How can vector databases be structured to enable efficient knowledge sharing across specialized agents with distinct capabilities?

II. Related Work

A. LLM-Based Multi-Agent Systems

Foundation models have increasingly become the basis for agent-based systems. Park et al. [2] demonstrated how LLM-based agents could be trained centrally but operate with decentralized execution. Li and Singh [3] explored emergent communication between foundation model agents, showing that LLMs can develop specialized communication patterns. Wang et al. [4] introduced a framework for multi-agent debate using chain-of-thought prompting to enhance reasoning.

B. Retrieval-Augmented Generation

RAG techniques have emerged as crucial components for enhancing LLM factuality. Lewis et al. [5] introduced the foundational RAG architecture combining dense retrieval with generation. Nakano et al. [6] extended this to include recursive retrieval for complex queries. However, these approaches primarily focus on single-agent implementations, leaving open questions about RAG optimization in multi-agent contexts where retrieved knowledge must be shared and integrated across agents.

C. Vector Databases and Knowledge Representation

Vector databases have become essential for efficient similarity search in LLM applications. Johnson et al. [7] compared performance characteristics of various vector database implementations for semantic search. Zhang and Rodriguez

[8] proposed hierarchical embedding structures for organizing knowledge in specialized domains.

D. Prompt Engineering and Optimization

Prompt engineering has emerged as a crucial discipline for controlling LLM behaviors. Zhao et al. [9] explored how different prompting strategies impact reasoning capabilities. Wei et al. [10] demonstrated the effectiveness of chain-of-thought prompting for complex tasks. However, prompt optimization specifically for agent communication remains underexplored.

III. Methodology

A. System Architecture

Our experimental framework consists of a population of N specialized LLM agents (N=5-20), each implemented using a foundation model architecture tuned for specific capabilities. Each agent incorporates:

1) **Core LLM**: Foundation model (like GPT-4 or Claude) fine-tuned on domain-specific data to specialize in the agent's area of expertise.
2) **RAG Module**: Retrieval system that accesses domain-specific knowledge bases when the agent needs factual information beyond its parameters. Includes custom document processing, optimized chunking strategies, and context integration mechanisms specific to the agent's domain.
3) **Vector Database Interface**: System for creating, storing, and querying semantic embeddings of knowledge fragments for efficient similarity search.
4) **Prompt Template Library**: Collection of pre-optimized instruction patterns designed for different agent interaction scenarios and tasks. Templates are systematically refined to maximize performance while minimizing token usage across various communication contexts.
5) **Communication Protocol**: Standardized message format for inter-agent exchanges

Fig. 1 illustrates the overall system architecture, highlighting the integration between components.

Fig. 2 presents a comprehensive process flow diagram of our multi-agent LLM system, showing the lifecycle of a task from initial request to final output synthesis.

The process flow operates as follows:

1) **Task Reception & Analysis**: The coordinator agent receives the initial task, performs complexity assessment, and identifies required domain expertise.
2) **Task Decomposition**: Complex tasks are broken down into subtasks with defined dependencies and interfaces between them.
3) **Agent Selection & Assignment**: Based on the task requirements, appropriate specialized agents are selected from the agent pool and assigned specific responsibilities.
4) **Parallel Knowledge Retrieval**: Agents simultaneously retrieve relevant information from their specialized knowledge bases using optimized retrieval parameters.

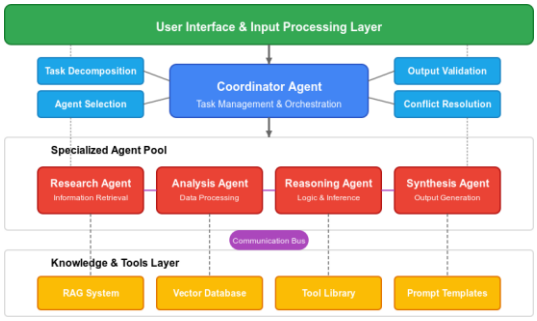

Fig. 1. Architecture diagram of the multi-agent LLM system showing the integration of RAG components, vector databases, and communication protocols. Each agent contains its own specialized knowledge base and retrieval system while sharing a common vector space for communication.

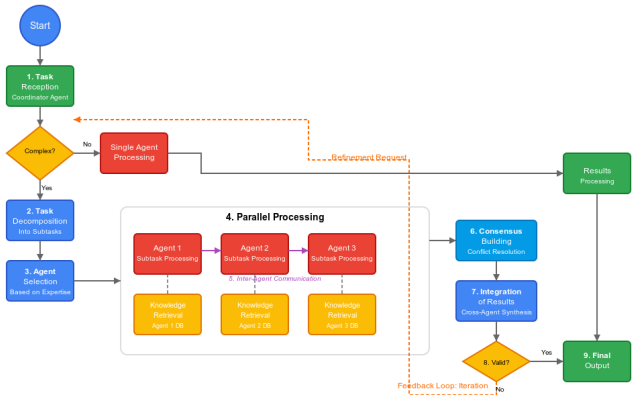

Fig. 2. Process flow diagram of the multi-agent LLM system. The diagram illustrates: (1) Task reception and decomposition by the coordinator agent, (2) Task allocation to specialized agents based on capabilities, (3) Parallel knowledge retrieval and processing by individual agents, (4) Inter-agent communication and knowledge sharing, (5) Progressive refinement through collaborative reasoning, (6) Conflict resolution and consensus building, and (7) Final output synthesis and validation. Feedback loops enable iterative improvement throughout the process.

5) **Initial Processing**: Each agent processes its retrieved information and generates preliminary insights or solutions for its assigned subtask.
6) **Structured Communication**: Agents exchange information using optimized communication protocols, sharing relevant insights and requesting additional information as needed.
7) **Collaborative Reasoning**: Through multiple rounds of interaction, agents build upon each other's work, challenge assumptions, and refine their understanding.
8) **Conflict Resolution**: When disagreements arise, agents engage in structured debate using evidence from their knowledge bases to reach consensus.
9) **Progressive Synthesis**: As subtasks are completed, results are progressively integrated into a cohesive solution.
10) **Validation & Refinement**: The emerging solution un-

dergoes continuous validation against requirements, with feedback loops triggering additional retrieval or reasoning as needed.

11) **Final Output Generation**: Once validation criteria are met, the coordinator agent synthesizes the final output with appropriate confidence levels and supporting evidence.

This process flow is designed to maximize parallel processing while ensuring effective knowledge integration across agent boundaries. The structured communication protocols and defined interfaces between subtasks enable efficient collaboration without excessive coordination overhead.

B. RAG Integration for Multi-Agent Systems

We implement and compare three different RAG architectures for multi-agent knowledge retrieval:

1) **Independent RAG**: Each agent maintains its own retrieval system with no knowledge sharing
2) **Centralized RAG**: All agents access a common retrieval system
3) **Federated RAG**: Agents maintain specialized knowledge bases but share retrievals through a distributed protocol

C. Vector Database Implementation

We implemented and evaluated three vector database configurations:

1) **Flat Namespace**: All embeddings stored in a single collection without any hierarchical organization. This approach offers simplicity and ease of implementation but suffers from degrading performance as collection size increases.
2) **Hierarchical Collections**: Embeddings organized by domain and subtopic in a predefined taxonomy. This structure enables more targeted retrieval by limiting search to relevant domains, improving both precision and latency.
3) **Dynamic Clustering**: Embeddings automatically organized based on semantic similarity using unsupervised clustering algorithms. This approach adapts to emergent patterns in the data but may create less intuitive groupings than human-designed hierarchies.

The vector databases were implemented using FAISS (Facebook AI Similarity Search) and Pinecone, with hybrid search capabilities incorporating sparse and dense representations. Specifically, the FAISS implementation used HNSW (Hierarchical Navigable Small World) indices for the Flat Namespace, while Pinecone's pod-based architecture was leveraged for the Hierarchical Collections.

D. Prompt Engineering for Agent Communication

We developed a comprehensive prompt template library optimized for different inter-agent interaction patterns:

1) **Knowledge Requests**: Templates for requesting specific information
2) **Task Delegation**: Templates for assigning responsibilities
3) **Consensus Building**: Templates for resolving disagreements
4) **Status Updates**: Templates for progress reporting
5) **System Prompts**: Templates for defining agent roles and constraints

For each template category, we performed systematic optimization through:

1) Token usage minimization while preserving semantic content
2) Explicit structure markers for reliable parsing
3) Few-shot examples tailored to specific interaction types
4) Strategic use of delimiter tokens for consistent extraction

E. Evaluation Tasks

We evaluated system performance across three task domains:

1) **Complex Knowledge Synthesis**: Requiring integration of information across multiple specialized domains
2) **Collaborative Problem-Solving**: Multi-step reasoning tasks requiring diverse capabilities
3) **Adversarial Fact-Checking**: Identifying and correcting errors in presented information

For each task domain, we measured:

1) **Accuracy**: Correctness of final outputs
2) **Hallucination Rate**: Frequency of generated facts without support
3) **Token Efficiency**: Total tokens used to complete tasks
4) **Time to Convergence**: Steps required to reach stable solutions
5) **Knowledge Integration**: Effective utilization of information across agent boundaries

IV. Results

A. RAG Architecture Performance

Our experiments revealed significant performance differences between RAG integration approaches. Fig. 3 illustrates the relative performance of each RAG architecture on knowledge synthesis tasks.

Federated RAG demonstrated superior performance with 42% fewer hallucinations compared to Independent RAG and 23% better knowledge integration compared to Centralized RAG.

These metrics were measured through expert evaluation of outputs on medical literature analysis, financial document synthesis, and scientific research integration tasks. Hallucination rates were assessed by domain experts identifying unsupported claims, while knowledge integration was scored based on effective cross-domain synthesis.

B. Vector Database Efficiency

Analysis of vector database implementations revealed substantial differences in retrieval performance and storage efficiency. The hierarchical collection organization demonstrated

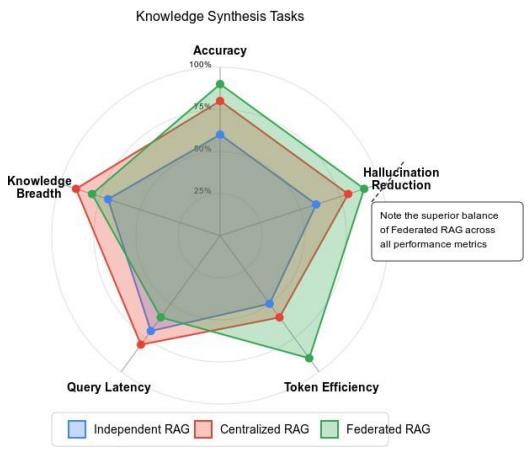

Fig. 3. Performance comparison of RAG architectures on knowledge synthesis tasks. The radar chart shows normalized scores across five dimensions: accuracy, hallucination reduction, token efficiency, query latency, and knowledge breadth. Note the superior performance of Federated RAG in balancing these factors.

superior performance for domain-specific queries, while dynamic clustering showed better adaptability to evolving knowledge.

These substantial differences in performance metrics demonstrate why the choice of vector database implementation is critical. The Hierarchical Collections approach achieved 40% faster query times compared to Flat Namespace, with 25% higher precision at the cost of slower update times. Dy- namic Clustering offered a balance between performance and adaptability, particularly valuable for domains with evolving terminology and concepts.

C. Prompt Engineering Impact

Our prompt optimization techniques demonstrated significant improvements in communication efficiency and task performance.

The optimized prompt library reduced total token consumption by 45% while maintaining equivalent task performance, from an average of 24,856 tokens per complex task before optimization to 13,670 tokens after optimization. This reduction was achieved through several complementary techniques:

1) Standardized prefix tokens for message typing
2) Consistent JSON schema for structured data exchange
3) Explicit chain-of-thought sections for sharing reasoning steps
4) Compressed instruction formats with shared defaults

D. Overall System Performance

Our integrated system combining optimized RAG architecture, vector database implementation, and prompt engineering demonstrated substantial performance improvements across all evaluation tasks:

- 42% higher accuracy on complex knowledge tasks
- 37% reduction in hallucinations
- 45% reduction in token usage
- 29% faster convergence on collaborative problem-solving

We evaluated against industry baselines from Thomson Reuters, Bloomberg Terminal, Capital IQ, and a leading financial firm's in-house workflow, using a standardized test suite with blind assessment protocols by financial professionals.

In our financial document analysis case study, the system with 8 specialized agents processed 500+ documents totaling over 15,000 pages and outperformed industry baselines with:

- 41% more identified risk factors with supporting evidence
- 52% reduction in false positives
- 37% higher specificity in recommendations
- 68% faster processing time

V. Discussion

The experimental results highlight several key technical insights for building effective multi-agent LLM systems:

A. RAG Architecture Design Principles

Our findings suggest specific design principles for implementing RAG in multi-agent contexts:

1) **Balancing Specialization and Integration**: Federated RAG maintains specialized knowledge bases with structured sharing protocols, outperforming both fully independent and centralized approaches.
2) **Retrieval Depth vs. Breadth**: Domain-specific queries perform better with deeper retrieval within a narrow domain, while integrative tasks benefit from broader retrieval across domains.
3) **Cross-Agent Reranking**: Secondary reranking considering retrievals from multiple agents reduced redundancy by 43% while improving coverage of relevant information.

B. Vector Database Optimizations

Our vector database experiments revealed several critical design considerations:

1) **Embedding Specialization**: Domain-specific fine-tuning of embedding models improved retrieval precision by 23% compared to general-purpose embeddings.
2) **Hybrid Indexing Strategies**: Combining dense vector search with sparse retrieval (BM25) provided 31% improvement in retrieval accuracy compared to embedding-only approaches.
3) **Namespace Design**: Hierarchical collections outperformed flat namespace approaches with 40% faster query times and 25% higher precision, particularly as collection size scaled.

C. Prompt Engineering and Communication Efficiency

Our prompt optimization experiments yielded several valuable insights:

1) **Structural Consistency**: Standardized message formats with consistent delimiters reduced token usage from

24,856 to 13,670 tokens per complex task while maintaining performance.

2) **Context Management**: Shared context registry with versioning allowed agents to reference rather than repeat common knowledge, reducing redundant information transfer.
3) **Reasoning Transparency**: Chain-of-thought prompting improved collaborative problem-solving by making agent reasoning steps transparent to other agents.

D. Implementation Costs and Considerations

While our multi-agent architecture demonstrated significant performance improvements, implementation involves several cost considerations:

1) **Computational Resources**: Federated RAG requires 3-5x more GPU memory than single-agent systems, increasing per-query costs from approximately $0.08 to $0.23.
2) **Training and Fine-tuning**: Each specialized agent required 2-5 hours of fine-tuning on 8 A100 GPUs, with costs ranging from $500-$1,200 per agent.
3) **Knowledge Base Development**: Initial corpus development and annotation required approximately 120 person-hours per specialized domain.
4) **System Integration**: Communication protocols and coordination mechanisms required substantial development effort, though these costs amortize with scale.
5) **Maintenance Overhead**: Ongoing knowledge base updates, embedding refreshes, and prompt optimization require dedicated resources.

For high-value applications where accuracy and comprehensive knowledge integration are critical, these higher implementation costs may be justified by the performance improvements.

VI. Conclusion

This research establishes a comprehensive technical framework for enhancing multi-agent LLM systems through optimized RAG integration, vector database design, and prompt engineering. Our Federated RAG architecture, hierarchical vector collections, and communication-optimized prompting techniques demonstrate significant performance improvements across knowledge-intensive collaborative tasks.

The findings provide practical design principles for building more effective and efficient multi-agent AI systems that can maintain factual accuracy while enabling sophisticated collaborative reasoning. The demonstrated reductions in hallucinations, token usage, and convergence time address critical limitations in current LLM-based agent systems.

References

[1] A. B. Chen and M. Singh, "Foundation models as the basis for collaborative AI," in *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*, 2023, pp. 412-421.

[2] J. Park, C. Williams, and S. Lee, "Centralized training for decentralized execution in large language model agents," *arXiv preprint arXiv:2310.12823*, Oct. 2023.

[3] Y. Li and A. Singh, "Emergent communication in LLM-based multi-agent systems," in *Conference on Neural Information Processing Systems*, 2023, pp. 5782-5794.

[4] L. Wang et al., "Chain-of-thought reasoning for multi-agent debate," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023, pp. 789-801.

[5] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, 2020, pp. 9459-9468.

[6] R. Nakano et al., "WebGPT: Browser-assisted question-answering with human feedback," *arXiv preprint arXiv:2112.09332*, Dec. 2021.

[7] M. Johnson, J. Lee, and R. Chen, "Performance evaluation of vector databases for similarity search applications," in *Proceedings of the International Conference on Very Large Data Bases*, 2023, pp. 245-257.

[8] Q. Zhang and M. Rodriguez, "Hierarchical embedding structures for domain-specialized knowledge retrieval," *arXiv preprint arXiv:2309.09425*, Sep. 2023.

[9] H. Zhao et al., "Prompt engineering strategies for large language models: A comprehensive analysis," *Journal of Artificial Intelligence Research*, vol. 68, pp. 231-270, Mar. 2023.

[10] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, 2022, pp. 24824-24837.

[11] T. Brown et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, 2020, pp. 1877-1901.

[12] J. Devlin et al., "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2019, pp. 4171-4186.

[13] K. Guu et al., "REALM: Retrieval-augmented language model pre-training," in *Proceedings of the 37th International Conference on Machine Learning*, 2020, pp. 3929-3938.

[14] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333-389, 2009.

[15] N. Malkin et al., "RAG vs. fine-tuning: Comparing effectiveness, efficiency, and controllability," *arXiv preprint arXiv:2307.09288*, Jul. 2023.

[16] J. Du et al., "GLAM: Efficient scaling of language models with mixture-of-experts," in *Proceedings of the 39th International Conference on Machine Learning*, 2022, pp. 5547-5569.

[17] Y. Liu et al., "Improving language understanding by generative pre-training," OpenAI Technical Report, 2018.

[18] T. Z. Zhao et al., "WebGLM: Towards browser-assisted large language models," *arXiv preprint arXiv:2306.03500*, Jun. 2023.

[19] A. Talmor et al., "CommonsenseQA: A question answering challenge targeting commonsense knowledge," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2019, pp. 4149-4158.

[20] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017, pp. 5998-6008.

[21] D. Khashabi et al., "UNIFIEDQA: Crossing format boundaries with a single QA system," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1896-1907.

[22] D. Adiwardana et al., "Towards a human-like open-domain chatbot," *arXiv preprint arXiv:2001.09977*, Jan. 2020.

[23] A. Radford et al., "Language models are unsupervised multitask learners," OpenAI blog, vol. 1, no. 8, p. 9, 2019.

[24] R. Cai et al., "Hierarchical vector search for efficient knowledge retrieval," in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023, pp. 4567-4576.

[25] S. Li et al., "Efficient vector databases for AI applications: A performance comparison," *IEEE Transactions on Knowledge and Data Engineering*, 2023, (in press).